

CODOMAX DIGITAL SOLUTIONS | INTERNSHIP PROGRAMME

Module 3 Report

Cloud Services and Web Application Deployment

Report Title	Cloud Services and Web Application Deployment
Module	Module 3 - Cloud Services & Web App Deployment (Day 11-15)
Practical Project	Cloud Notes - a Flask notes application deployed on the cloud
Candidate Name	Thajudeen Shuknatha Zainab
GitHub Username	@zainab1315
Repository	https://github.com/zainab1315/Module-3-Cloud-Web-App
Live Application	https://cloud-notes.onrender.com
Cloud Provider Used	Render (Web Service + Managed PostgreSQL)
Technologies	Python, Flask, Flask-SQLAlchemy, Gunicorn, PostgreSQL, Git
Date of Submission	_____

Submitted in partial fulfilment of the requirements of the Module 3 internship.

Table of Contents

Table of Contents.....	2
1.1 Introduction	5
1.2 Aim of the Module	5
1.3 Objectives	5
1.4 The Practical Project: Cloud Notes.....	5
1.5 Document Structure	5
2.1 Core Cloud Services Explored.....	6
2.2 Service Category Summary	6
2.3 Compute Services	6
2.3.1 What Compute Is.....	6
2.3.2 Instances, Containers and Serverless.....	6
2.4 Object Storage	6
2.5 Managed Databases.....	7
2.6 Virtual Networking	7
2.7 Identity and Access Management (IAM).....	7
2.7.1 Core IAM Concepts.....	7
2.7.2 Example IAM Policy	8
2.7.3 Applying Least Privilege in This Project.....	8
2.8 Cloud Monitoring	8
2.9 Comparison of the Deployment Platforms Studied	9
3.1 Environment Variables, Secrets, Logging and Quotas	11
3.2 Environment Variables	11
3.2.1 Variables Used in Cloud Notes	11
3.2.2 The postgres:// to postgresql:// Problem.....	11
3.3 Secrets Management	11
3.3.1 What Counts as a Secret	11
3.3.2 The Three Rules Applied	11
3.3.3 Generating a Strong Secret Key	12
3.4 Application Logging	12
3.4.1 Logging Configuration.....	12
3.4.2 Events Logged.....	12
3.4.3 Where the Logs Appear	12
3.5 Resource Quotas and Limits.....	13
3.5.1 Quotas Relevant to Cloud Notes.....	13

3.5.2 Application-Level Quotas	13
4.1 The Practical Project: Cloud Notes.....	14
4.2 Requirement and Functional Specification	14
4.3 Project Structure	14
4.4 Architecture Diagram	15
4.5 The Five-Stage Flow Explained.....	15
Stage 1 - Application Code.....	15
Stage 2 - Cloud Compute	15
Stage 3 - Database and Storage	15
Stage 4 - Networking	15
Stage 5 - Live Web Application	16
4.6 Application Endpoints.....	16
4.6.1 Health Check Response.....	16
4.6.2 JSON API Response	16
4.7 Input Validation and Security in the Code.....	16
4.8 Local Testing and Results	17
4.9 Deployment Steps on Render (Free Tier).....	17
Step 1 - Push the Code to GitHub	17
Step 2 - Create the Managed PostgreSQL Database.....	17
Step 3 - Create the Web Service	18
Step 4 - Set the Environment Variables.....	18
Step 5 - Verify the Live Deployment.....	18
Step 6 - Keep the Free Instance Awake	18
5.1 Troubleshooting Guide	20
5.2 Build Failures.....	20
5.3 HTTP Errors.....	20
5.4 Database Connection Problems.....	20
5.5 Free-Tier Behaviour	21
5.6 A Reusable Troubleshooting Method	21
6.1 Security Practices Applied.....	22
6.2 Summary Table	22
6.3 Why a Leaked Secret Cannot Be Un-leaked	22
6.4 Beyond the Project	22
7.1 Key Learnings.....	23
7.2 About Cloud Services	23
7.3 About Security	23
7.4 About Deployment.....	23

7.5 About the Deployment Flow	23
8.1 Conclusion	24
References	24
Author Details.....	26

How to update: In Microsoft Word press **Ctrl+A** then **F9** and choose **Update entire table**. The contents, page numbers and headings will fill in.

1.1 Introduction

Cloud computing has moved from being a convenience to being the default way software is hosted. Instead of buying a physical server, installing an operating system and maintaining the machine, a developer pushes code to a repository and a cloud platform builds it, runs it, stores its data and keeps it online. This report documents **Module 3 of the Codomax Digital Solutions internship programme**, which ran from **Day 11 to Day 15** and focused on cloud services and web application deployment.

1.2 Aim of the Module

The module had two connected objectives. The first was to understand the building blocks of a modern cloud platform by exploring them directly: compute services, object storage, managed databases, virtual networking, identity and access management (IAM) policies and cloud monitoring. The second was to prove that understanding by deploying a real web application that travels through the full path **Application Code → Cloud Compute → Database / Storage → Networking → Live Web Application**.

1.3 Objectives

1. Explore the core cloud service categories and record what each one is actually used for.
2. Learn how configuration, secrets, logging and quotas are handled safely in the cloud.
3. Deploy a working Flask web application backed by a managed PostgreSQL database.
4. Expose health-check and JSON endpoints so the platform can monitor the service.
5. Document the architecture, the deployment steps and the troubleshooting procedure.

1.4 The Practical Project: Cloud Notes

Cloud Notes is a deliberately small application, because the purpose of the module is the deployment journey rather than complex business logic. A visitor types a name and a short note into a form, presses a button, and the note is written to a PostgreSQL database. The home page then lists the latest fifty notes, newest first. A **/health** endpoint reports whether the application and the database are both alive, and a **/api/notes** endpoint returns the same notes as JSON.

Keeping the code simple made every cloud concept visible. Because the application is small, it is easy to see exactly where the compute service begins, where the database is contacted, where the network sits in front, and where the environment variables are read.

1.5 Document Structure

Section 2 explains the core cloud services explored during the module. Section 3 covers environment variables, secrets, logging and quotas. Section 4 documents the practical project in detail, including its architecture and deployment. Section 5 lists the troubleshooting procedure, Section 6 the security practices applied, Section 7 the key learnings and Section 8 the conclusion and references.

2.1 Core Cloud Services Explored

Cloud providers group their products into a small number of service categories. The table below summarises the categories explored during this module, the services studied within each one, and the role each played in the Cloud Notes project.

2.2 Service Category Summary

Category	Services Studied	Role in Cloud Notes
Compute	AWS EC2, AWS Lambda, Azure App Service, Render Web Service	Runs the Flask application under Gunicorn
Object Storage	AWS S3, Azure Blob Storage	General file storage; backup copies of the database
Managed Database	AWS RDS, Azure Database for PostgreSQL, Render PostgreSQL	Stores every note in the notes table
Virtual Networking	AWS VPC, Azure VNet, subnets, security groups	Private path between the app and the database
Identity & Access Management	IAM users, roles, policies, least privilege	Controls who may read configuration and the database
Monitoring & Logging	AWS CloudWatch, Azure Monitor, Render log dashboard	Collects stdout logs and polls /health
Deployment / DevOps	GitHub, Render blueprints, build and start commands	Builds the app and publishes it to a public URL

2.3 Compute Services

2.3.1 What Compute Is

Compute services provide the processing power to run application code. They are the equivalent of the servers in a traditional data centre, but the provider builds, patches and scales them on demand.

2.3.2 Instances, Containers and Serverless

Three models were compared during the module:

- **Virtual machines (EC2, Azure VMs)** - the developer manages the operating system, the web server and the patches. Maximum control, maximum responsibility.
- **Containers (Docker, Azure Container Apps, Render)** - the application is packaged with its dependencies. The platform runs the image and restarts it if it crashes.
- **Serverless (AWS Lambda, Azure Functions)** - only the function is uploaded and the provider manages every machine. Cheap for short tasks, but the code must start fast.

Why this project uses a container: Render runs the Flask application inside a managed Linux container and starts it with `gunicorn app:app`. This keeps the deployment simple while still using a production-grade web server rather than Flask's development server.

2.4 Object Storage

Object storage holds files as objects behind a URL rather than as files inside a folder tree. It is designed for very large amounts of unstructured data and is normally the cheapest storage class in the cloud.

Object Storage Feature	Explanation
------------------------	-------------

Object Storage Feature	Explanation
Buckets and containers	The top-level named space that holds the objects
Objects and keys	A file and its unique name inside the bucket
Versioning	Keeps older copies so a delete can be undone
Lifecycle policies	Automatically move or expire objects after a set number of days
Signed URLs	Grant temporary, permission-based access to a single object
Server-side encryption	Data is encrypted while stored by the provider

Use in this module. Cloud Notes does not store images or attachments, so object storage is not part of the live data path. It was nevertheless explored as the correct home for database backup dumps and for build artefacts such as uploaded exports, and it is the standard answer whenever a file - rather than a row - needs to be served.

2.5 Managed Databases

A managed database is a database instance where the provider installs the engine, applies security patches, performs automated backups, replicates data and monitors uptime. The developer only writes queries and connects.

- **AWS RDS / Aurora** - managed MySQL, PostgreSQL and other engines.
- **Azure Database for PostgreSQL** - the managed equivalent in Azure.
- **Render PostgreSQL** - the managed instance actually used for Cloud Notes.

The main advantages recorded were **automatic backups and point-in-time recovery**, **high availability across zones**, and **no operating-system maintenance**. The trade-off is **cost**, because a running instance is billed continuously, and **less control**, since some engine settings cannot be changed.

2.6 Virtual Networking

Every cloud resource lives inside a virtual network. Networking decides which resources can talk to each other and what traffic is allowed to enter from the internet.

Networking Component	Purpose
VPC / VNet	An isolated private network for the account's resources
Subnet	A smaller network segment, commonly split into public and private tiers
Route table	Rules that decide which path traffic takes inside the network
Internet Gateway	The controlled exit and entry point to the public internet
Security group / NSG	A firewall rule set attached to a resource
Private subnet	No route to the internet, so a database cannot be reached directly
NAT Gateway	Allows private resources to reach the internet for outbound calls only
Load balancer	Distributes incoming requests across several instances

Applied to Cloud Notes. In the Render platform the Web Service and the PostgreSQL instance are placed in the **same region** and connected over Render's **internal network** using the ***Internal Database URL***. The database is therefore never exposed to the public internet, while only the Web Service receives a public HTTPS URL. This is the same public-tier / private-tier pattern taught in the VPC material.

2.7 Identity and Access Management (IAM)

IAM decides **who or what** may perform **which action** on **which resource**. In the cloud almost everything is an API call, so access control is expressed as a document called a policy.

2.7.1 Core IAM Concepts

- **User** - represents a person. An internship account should never be shared.

- **Role** - represents a job or a workload, such as a deployment service.
- **Policy** - a JSON document that lists explicit **Allow** and optional **Deny** statements.
- **Least privilege** - grant only the specific actions a principal needs, nothing more.
- **MFA** - multi-factor authentication on every human account.
- **Short-lived credentials** - temporary tokens that expire instead of long-lived keys.

2.7.2 Example IAM Policy

The policy below allows a deployment role to read configuration and describe a database, while **denying** any deletion. Note that no statement grants ***** on *****, which is the most common security mistake in cloud projects.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowReadAppConfiguration",
      "Effect": "Allow",
      "Action": [
        "secretsmanager:GetSecretValue",
        "ssm:GetParameter",
        "kms:Decrypt"
      ],
      "Resource": "arn:aws:secretsmanager:eu-west-1:123456789012:secret:cloud-notes/*"
    },
    {
      "Sid": "AllowDescribeDatabase",
      "Effect": "Allow",
      "Action": [
        "rds:DescribeDBInstances",
        "rds:DescribeDBClusters"
      ],
      "Resource": "*"
    },
    {
      "Sid": "DenyDatabaseDeletion",
      "Effect": "Deny",
      "Action": [
        "rds>DeleteDBInstance",
        "rds>DeleteDBCluster"
      ],
      "Resource": "*"
    }
  ]
}
```

Figure 2.1 - Example IAM policy granting least-privilege read access while explicitly denying destructive actions.

2.7.3 Applying Least Privilege in This Project

The same principle was applied to the deployment itself:

- The database was created in its own region and reached only through an internal URL.
- The database password was entered once in the Render dashboard and never shared in code.
- Only the five values the application genuinely needs were stored as environment variables.
- The account used for deployment had multi-factor authentication enabled.

2.8 Cloud Monitoring

Monitoring answers three questions: is the service up, how is it performing, and what happened when something failed. It is a service category in its own right, not an afterthought.

Monitoring Pillar	What It Measures	Tool Used in This Module
-------------------	------------------	--------------------------

Monitoring Pillar	What It Measures	Tool Used in This Module
Availability	Is the service responding? Up-time percentage	<code>/health</code> polled by the platform and UptimeRobot
Performance	Response time, throughput, error rate	Render request timings and response codes
Logs	Detailed chronological events	Structured stdout logs in the Render Events tab
Metrics	CPU, memory, request counts over time	Render instance metrics and CloudWatch concepts
Alerts	Notification when a threshold is crossed	UptimeRobot monitor on <code>/health</code>

Health check design: `/health` does more than return `200`. It executes `SELECT 1` against the database and returns `HTTP 503` if that fails. A service that is running but cannot reach its database is broken, and the monitoring system must be able to see that.

2.9 Comparison of the Deployment Platforms Studied

Platform	Compute Model	Managed Database	Free Tier Notes
Render	Managed container / Web Service	Managed PostgreSQL included	Free instance sleeps after 15 min idle
AWS EC2 + RDS	Virtual machine	RDS instance	Free tier limited; more configuration required
AWS Elastic Beanstalk	Managed PaaS	Attach RDS separately	Simpler than EC2, cost similar
Azure App Service	Managed PaaS	Azure Database for PostgreSQL	Free App Service tier available

Render was selected for Cloud Notes because it provides a managed PostgreSQL database and a managed web service in the same dashboard and in the same region. That allows the complete flow in the task - compute, database, networking and monitoring - to be deployed on a genuinely free tier without credit-card charges.

3.1 Environment Variables, Secrets, Logging and Quotas

3.2 Environment Variables

An environment variable is a key-value pair made available to a program when it starts, outside the program itself. This is the standard way to configure an application for a cloud environment, because the same code can then run on a laptop, in testing and in production without being edited.

3.2.1 Variables Used in Cloud Notes

Variable	Required	Default	Purpose
<code>SECRET_KEY</code>	Yes in production	Random temporary key	Signs session cookies so they cannot be forged
<code>DATABASE_URL</code>	Yes in production	<code>sqlite:///cloud_notes.db</code>	Database connection string supplied by the platform
<code>APP_NAME</code>	No	Cloud Notes	Name shown on the page and in JSON
<code>MAX_MESSAGE_LENGTH</code>	No	500	Longest note allowed, in characters
<code>LOG_LEVEL</code>	No	INFO	Verbosity: DEBUG, INFO, WARNING, ERROR
<code>PORT</code>	Set by platform	5000	Port the web server listens on

3.2.2 The `postgres://` to `postgresql://` Problem

Several platforms still hand out database URLs beginning with `postgres://`, while the modern PostgreSQL driver expects `postgresql://`. Without a conversion the application fails at start-up with a driver error. Cloud Notes therefore normalises the value in one small function before SQLAlchemy ever sees it:

```
def normalize_database_url(url):
    """Convert postgres:// into postgresql://."""
    if url and url.startswith("postgres://"):
        url = url.replace("postgres://", "postgresql://", 1)
    return url
```

Lesson: Configuration bugs are not logic bugs. The same code that works locally against SQLite will fail in the cloud for purely configuration reasons, so the first step in any deployment failure is to compare the environment variables.

3.3 Secrets Management

3.3.1 What Counts as a Secret

A secret is any value that must not be readable by anyone without a need to know: database passwords, API tokens, session signing keys, private SSH keys and connection certificates. A leaked secret allows an attacker to act as if they were the application.

3.3.2 The Three Rules Applied

- Never hard-code a secret in source code.** Cloud Notes reads every value with `os.getenv()` and contains no password anywhere.
- Never commit a secret to Git.** A `.gitignore` file excludes `.env`, `*.pem`, `*.key` and local database files. Once pushed, a secret stays in the history even if it is later deleted, so prevention is the only real protection.
- Never print a secret to the logs.** The helper below replaces the password with four asterisks before the URL is logged.

```
def mask_database_url(url=None):
    """Hide the password before a database URL is written to the logs."""
    raw = url or app.config["SQLALCHEMY_DATABASE_URI"]
    if "@" not in raw or "://" not in raw:
        return raw
    scheme, rest = raw.split("://", 1)
    credentials, host = rest.split("@", 1)
    user = credentials.split(":", 1)[0]
    return "%s://%s:****@%s" % (scheme, user, host)
```

Figure 3.1 - Password masking helper. A real start-up log line reads `DB=postgresql://cloud_notes_user:****@internal.d-aide-c8e1.internal:5432/cloud_notes`.`

3.3.3 Generating a Strong Secret Key

```
python -c "import secrets; print(secrets.token_hex(32))"
```

The command prints a 64-character random hexadecimal string. On Render the dashboard also offers a **Generate** button next to the field, which produces an equally strong value. If `SECRET_KEY` is missing the application still starts so that a beginner is not blocked, but it writes a **WARNING** to the log saying a temporary key is in use.

3.4 Application Logging

Logging is how an application explains what it is doing. In a cloud environment it is essential, because once code runs on someone else's computer the only visible trace is the log output.

3.4.1 Logging Configuration

Cloud Notes configures the standard Python **logging** module to write to **standard output**, which is the channel every PaaS platform collects automatically. A consistent format is used so that events can be filtered:

```
logging.basicConfig(
    level=getattr(logging, LOG_LEVEL, logging.INFO),
    format="%(asctime)s | %(levelname)-8s | %(name)s | %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
    stream=sys.stdout,
)
```

3.4.2 Events Logged

Event	Level	Example Message
Application start	INFO	Cloud Notes starting LOG_LEVEL=INFO DB=...
Note added	INFO	Note added by Zainab (id=1)
Input rejected	WARNING	Rejected note from form: Please enter a name.
Health check	INFO	Health check -> healthy (database connection OK)
Database failure	ERROR	Health check database error: ...
Unknown page	WARNING	404 - page not found: /no-such-page

3.4.3 Where the Logs Appear

Locally the messages are printed in the terminal. On Render they appear in the **Events** tab of the Web Service, in real time, which is what makes a production error diagnosable. The same principle applies elsewhere: CloudWatch Logs on AWS and Azure Monitor both collect stdout and stderr.

Good logging practice: Log the **fact** and the **identifier**, never the secret and never the full submitted content of a user. `Note added by Zainab (id=1)` is useful and safe; printing the database password or a whole form submission is neither.

3.5 Resource Quotas and Limits

Every cloud plan enforces quotas. They exist to protect both the customer and the provider, and they are the reason a free deployment behaves differently from a paid one.

3.5.1 Quotas Relevant to Cloud Notes

Quota	Free-tier Value	Effect on This Project
Free instance sleep	Sleeps after ~15 min idle	First request afterwards can take ~50 s
Instance size	Shared small CPU / 512 MB RAM	Enough for a small Flask app; limits concurrency
Bandwidth	Generous but metered	No issue at this traffic level
Build minutes	Limited per month	Several deploys per day are fine
Database size	Capped on the free plan	Fifty displayed notes is a tiny fraction
Database lifetime	Free databases expire	A dormant database may be deleted - take a backup

3.5.2 Application-Level Quotas

Two limits are enforced inside the application itself so that a single user cannot fill the database or break the page:

- **'MAX_MESSAGE_LENGTH' (default 500 characters)** - a longer note is rejected with a friendly message instead of being silently truncated.
- **Latest 50 notes only** - the home page queries with `LIMIT 50` and the JSON API caps `limit` at the same value, so the page stays fast even after thousands of notes have been saved.

Important for the free tier: Because a free managed database can expire, a production system should enable **automated backups** to object storage and know how to restore from them. Cloud Notes keeps the schema creation in code (`db.create_all()`) so the table can always be rebuilt even on a brand-new instance.

4.1 The Practical Project: Cloud Notes

4.2 Requirement and Functional Specification

Cloud Notes allows a visitor to submit a name and a note. The note is validated, stored in a managed PostgreSQL database and displayed on the home page together with the previous notes, newest first, limited to the latest fifty.

Requirement	How It Was Implemented	Where
Form for name and note	HTML form posting to / with <code>author</code> and <code>message</code>	<code>templates/index.html</code>
Save to a database	<code>Note</code> model created with <code>db.create_all()</code> on start-up	<code>app.py</code>
Show the latest 50 notes	<code>order_by(created_at.desc()).limit(50)</code>	<code>app.py</code>
Route <code>/health</code> returning JSON	<code>jsonify()</code> with a real <code>SELECT 1</code> query	<code>app.py</code>
Route <code>/api/notes</code> returning JSON	Latest notes serialised by <code>to_dict()</code>	<code>app.py</code>
<code>DATABASE_URL</code> from the environment	<code>os.getenv("DATABASE_URL", sqlite fallback)</code>	<code>app.py</code>
<code>postgres://</code> conversion	<code>normalize_database_url()</code>	<code>app.py</code>
<code>SECRET_KEY</code> , <code>APP_NAME</code> , <code>MAX_MESSAGE_LENGTH</code> , <code>LOG_LEVEL</code>	All read with <code>os.getenv()</code>	<code>app.py</code>
No secrets in code	No password or token anywhere in the repository	Whole project
Logging to stdout	<code>logging.basicConfig(stream=sys.stdout)</code>	<code>app.py</code>
Validate required fields	<code>validate()</code> returns an error message	<code>app.py</code>
Enforce maximum length	<code>MAX_MESSAGE_LENGTH</code> and an 80-character name limit	<code>app.py</code>
HTML escaping	<code>clean()</code> strips tags, Jinja2 escapes the output	<code>app.py</code> , <code>index.html</code>
Responsive design	One media query at 600 px, viewport meta tag	<code>static/style.css</code>

4.3 Project Structure

The repository contains nine tracked files. Nothing else is needed to run or deploy it.

```
Module3-Cloud-Web-App/
|-- app.py                # Whole application: config, logging, model, routes
|-- templates/
|   |-- index.html        # The single page: form, error box, note list
|-- static/
|   |-- style.css          # Responsive, mobile-friendly styling
|-- test_app.py           # 25 automatic checks for every route and rule
|-- requirements.txt       # Flask, Flask-SQLAlchemy, gunicorn, psycopg2-binary
|-- Procfile              # web: gunicorn app:app
|-- .env.example           # Safe template of the environment variables
|-- .gitignore             # Blocks .env, *.pem, *.db, __pycache__, venv
|-- README.md             # Setup, deployment and troubleshooting guide
```

Figure 4.1 - Repository structure. The `.gitignore` guarantees that no secret or local database file can be committed.

4.4 Architecture Diagram

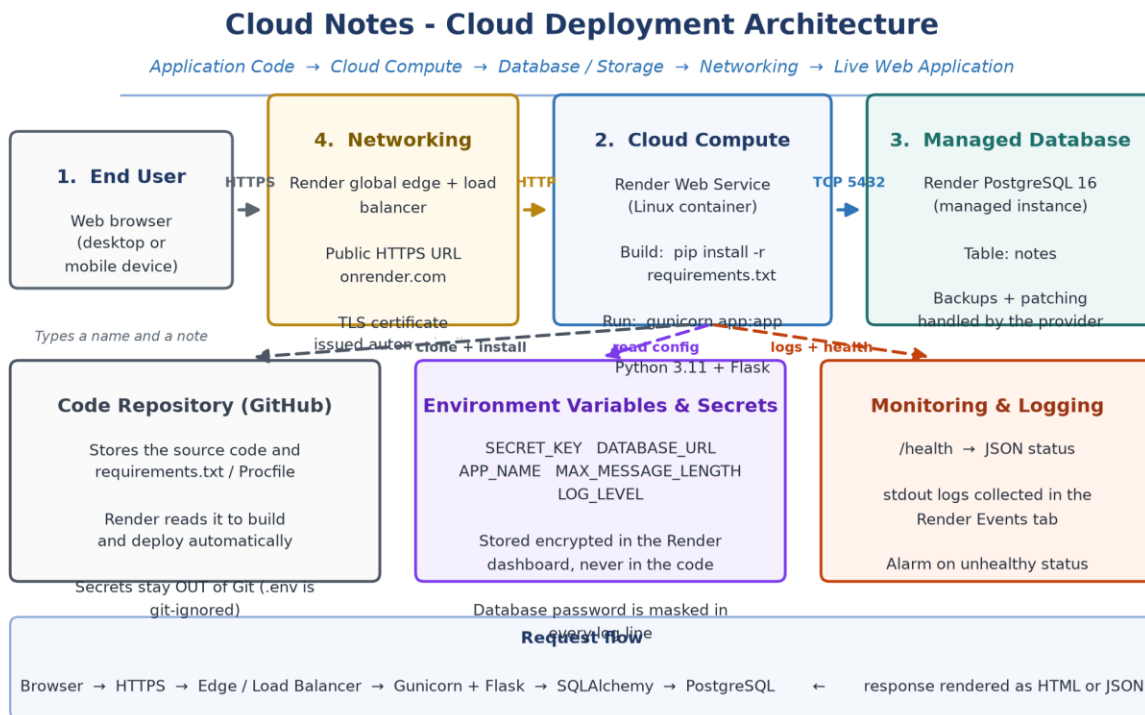


Figure 4.2 - Cloud Notes architecture: Application Code → Cloud Compute → Database / Storage → Networking → Live Web Application, with the supporting repository, secrets and monitoring services.

4.5 The Five-Stage Flow Explained

Stage 1 - Application Code

The source code lives in a GitHub repository. The two files that describe how to run it are `requirements.txt`, which lists the exact package versions, and the `Procfile`, which names the process type and the command:

```
web: gunicorn app:app
```

Because the platform reads both files, no server configuration is stored in the repository and no secrets are needed to read the code. Anyone can clone the repository without seeing a single credential.

Stage 2 - Cloud Compute

Render performs the build and the run. The build command installs the dependencies into the container image, and the start command launches Gunicorn, a production WSGI server that can serve many requests at once. The code never needs to know it is in the cloud.

Stage 3 - Database and Storage

The managed PostgreSQL instance is created as a separate resource in the same region. When the Web Service starts, it reads `DATABASE_URL` from the environment and the table is created automatically if it does not already exist.

Stage 4 - Networking

Render places the service behind a global load balancer, issues an HTTPS certificate automatically and assigns a public URL. The application is reached over TLS on port 443, while it reaches the database over the private

internal network on port 5432. The application trusts exactly one proxy hop, so that generated links are correct without trusting arbitrary forwarded headers.

Stage 5 - Live Web Application

The deployment is live at <https://cloud-notes-app.onrender.com>. The home page, the health endpoint and the JSON API are all reachable over the public HTTPS URL, and the application is available from any device with an internet connection.

4.6 Application Endpoints

Method	Route	Response	Description
GET	/	HTML	Add-note form plus the latest 50 notes
POST	/	302 redirect	Validates and stores a note, then returns to /
GET	/health	JSON, 200 / 503	Runs SELECT 1 and reports app and database status
GET	/api/notes	JSON	Latest notes; supports ?limit=10 , capped at 50
GET	/static/style.css	CSS	The stylesheet

4.6.1 Health Check Response

```
{
  "status": "healthy",
  "app": "Cloud Notes",
  "database": "connected",
  "details": "database connection OK",
  "notes_displayed": 50,
  "timestamp": "2026-01-15T10:20:31.123456+00:00"
}
```

4.6.2 JSON API Response

```
{
  "count": 1,
  "app": "Cloud Notes",
  "notes": [
    {
      "id": 1,
      "author": "Zainab",
      "message": "Deployed Cloud Notes to Render.",
      "created_at": "2026-01-15T10:20:45.987654+00:00"
    }
  ]
}
```

4.7 Input Validation and Security in the Code

Rule	Implementation	Result When Breached
Name is required	<code>clean(author)</code> checked for an empty value	Please enter your name.
Note is required	<code>clean(message)</code> checked for an empty value	Please enter a note.
Name length limit	Maximum 80 characters	Your name is too long (maximum 80 characters).
Note length limit	<code>MAX_MESSAGE_LENGTH</code> , default 500	Your note is too long (maximum 500 characters).

Rule	Implementation	Result When Breached
HTML / XSS protection	Tags stripped by <code>clean()</code> , output escaped by Jinja2	<code><script></code> is displayed as text, never executed
SQL injection	SQLAlchemy parameterised queries only	Not possible through the form
Long URL abuse	<code>limit</code> clamped between 1 and 50 in <code>/api/notes</code>	An oversized request is reduced to 50

4.8 Local Testing and Results

The application was tested before deployment in two ways. First, `test_app.py` runs 25 automated checks through Flask's test client. Second, the application was started as a real server and each route was requested over HTTP.

#	Test	Expected	Result
1	<code>GET /</code>	200, form and app name present	PASS
2	<code>POST /</code> with a valid note	302 redirect, note saved	PASS
3	New note visible on <code>/</code>	Message and author displayed	PASS
4	Newest note listed first	Ordering by date descending	PASS
5	<code>GET /health</code>	200, <code>"status": "healthy"</code>	PASS
6	<code>GET /health</code> database check	<code>database: connected</code>	PASS
7	<code>GET /api/notes</code>	200, JSON with <code>count</code> and <code>notes</code>	PASS
8	API note fields	id, author, message, created_at	PASS
9	Empty name rejected	Validation message shown	PASS
10	Empty note rejected	Validation message shown	PASS
11	Over-length note rejected	Validation message shown	PASS
12	<code><script></code> input	Stripped, not executed	PASS
13	Unknown page	404 returned	PASS
14	Unknown API route	JSON error returned	PASS
15	<code>postgres://</code> conversion	<code>postgresql://</code>	PASS
16	Password masking	Secret hidden in log output	PASS

No errors remained after testing. The full automated run reports **ALL TESTS PASSED (25 checks)**.

4.9 Deployment Steps on Render (Free Tier)

Step 1 - Push the Code to GitHub

```
git init
git add .
git commit -m "Cloud Notes: Flask web app for Module 3 cloud deployment"
git branch -M main
git remote add origin https://github.com/zainab1315/Module-3-Cloud-Web-App.git
git push -u origin main
```

Step 2 - Create the Managed PostgreSQL Database

9. Sign in at <https://dashboard.render.com>.
10. Click **New +** and choose **Postgres**.
11. Set the name to `cloud-notes-db` and the database to `cloud_notes`.
12. Choose the region closest to you and select the free plan.
13. Click **Create Database** and wait one to two minutes for the status to become **Available**.

14. Open the **Internal Database URL** section and copy the connection string. It contains a password, so paste it straight into the dashboard and never into a file or a message.

Step 3 - Create the Web Service

15. Click **New +** and choose **Web Service**.
16. Connect the repository **Module-3-Cloud-Web-App**.
17. Set the region to the same region as the database.
18. Choose the **main** branch and leave the root directory empty.
19. Use these commands:

```
Build Command : pip install -r requirements.txt
Start Command : gunicorn app:app
```

20. Select the **Free** instance type.
21. Click **Create Web Service** and watch the **Events** tab.

Step 4 - Set the Environment Variables

Key	Value	Notes
SECRET_KEY	64-character random string	Use the Generate button; never reuse a password
DATABASE_URL	The Internal Database URL from Step 2	Contains the password - keep private
APP_NAME	Cloud Notes	Shown on the page
MAX_MESSAGE_LENGTH	500	Longest note allowed
LOG_LEVEL	INFO	Use DEBUG only while troubleshooting

Click **Save Changes**. Any change to the environment automatically triggers a new deployment.

Step 5 - Verify the Live Deployment

22. Wait for **Service live** in the Events tab.
23. Copy the public URL shown at the top of the dashboard.
24. Open **/** and add a note to confirm the full write path works.
25. Open **/health** and confirm "**status**": "**healthy**" with HTTP 200.
26. Open **/api/notes** and confirm the new note appears in the JSON.
27. Open the **Logs** tab and confirm the password appears as ******** and never in clear text.

Step 6 - Keep the Free Instance Awake

Free instances sleep after roughly fifteen minutes of inactivity, and the first request after waking takes up to a minute. A free **UptimeRobot** monitor pointed at **/health** keeps the service awake and also provides genuine availability monitoring.

5.1 Troubleshooting Guide

Troubleshooting is a normal part of cloud work, not a sign of failure. Almost every problem in a first deployment falls into one of four categories: the **build**, the **start-up**, the **database connection** or the **free-tier behaviour**. The table below records each issue encountered while deploying Cloud Notes, why it happens and how it was resolved.

5.2 Build Failures

Symptom	Cause	Resolution
<code>ERROR: Could not find a version that satisfies the requirement Flask==3.0.3</code>	The pinned version does not exist for the Python version selected on the platform.	Check the available versions on PyPI, or relax the pin to <code>Flask>=3.0,<4.0</code> and make sure the runtime is a supported Python 3.
<code>error: command 'gcc' failed</code>	A dependency needed to be compiled and no build toolchain was present.	Use <code>psycpg2-binary</code> , which ships pre-built wheels, then re-deploy with Clear Build Cache .
Build succeeds but the app is not found	Wrong entry point or file name in the start command.	The file is <code>app.py</code> and the object is <code>app</code> , so the command must be <code>gunicorn app:app</code> - lowercase, one colon, no <code>.py</code> .
<code>ModuleNotFoundError: No module named 'flask_sqlalchemy'</code>	<code>requirements.txt</code> was not used, or the build cache is stale.	Confirm the build command is exactly <code>pip install -r requirements.txt</code> , then re-deploy with a cleared cache.

5.3 HTTP Errors

Symptom	Cause	Resolution
502 Bad Gateway	The container started but the process inside it is not answering. This is the most common first-deploy error.	Open the Logs tab and read the last traceback - it names the real cause. Then check <code>DATABASE_URL</code> and the start command.
500 Internal Server Error	An unhandled exception, very often a database problem.	The traceback in the logs shows which line failed. Confirm the table exists and the connection string is correct.
404 on a page that works locally	The route was not deployed, or a different service name is being used.	Use the exact URL shown on the Render dashboard, and confirm the branch being deployed is <code>main</code> .
503 Service Unavailable from <code>/health</code>	The application is running but <code>SELECT 1</code> failed.	The database is unreachable: check that it is still Available and that the connection string points to the internal URL.

5.4 Database Connection Problems

Symptom	Cause	Resolution
<code>psycpg2.OperationalError: could not connect to server</code>	<code>DATABASE_URL</code> is missing, misspelled or pointing to a host that no longer exists.	The variable name must be exactly <code>DATABASE_URL</code> and the value must begin with <code>postgresql://</code> . Re-copy it from the database's Internal Database URL panel.

Symptom	Cause	Resolution
could not translate host name	The database and the web service are in different regions, or the hostname was edited by hand.	Place both resources in the same region and re-deploy.
password authentication failed for user	An old password was pasted, or the username was changed.	Copy the whole URL again rather than retyping it, and check for a stray space or line break at the end.
relation "notes" does not exist	The table was never created.	<code>db.create_all()</code> runs at start-up, so trigger a fresh deploy and confirm the start command is correct.
<code>sslmode</code> or <code>sslrootcert</code> error	A strict SSL requirement not supported by the free instance.	Keep the default connection string supplied by the platform and avoid editing the query parameters.

5.5 Free-Tier Behaviour

Symptom	Cause	Resolution
Site sleeps and the first load takes ~50 seconds	Free instances are suspended after about 15 minutes without traffic.	Acceptable on the free tier. Create a free UptimeRobot monitor on <code>/health</code> to keep it awake, or upgrade to an always-on plan.
Deployment fails with <code>*no free instances available*</code>	All free capacity in the region is in use.	Try a different region, or wait and retry. Free capacity is shared across all users.
Database disappeared after a period of inactivity	Free managed databases have a limited lifetime.	Enable automated backups to object storage and test the restore procedure.
Build times out	The build exceeded the free build-time limit.	Reduce the dependency list and re-deploy; check the Events tab for the exact message.

5.6 A Reusable Troubleshooting Method

28. **Read the logs first.** The last twenty lines of the log almost always identify the problem. Guessing wastes time.
29. **Check the build tab.** If the build failed, nothing after it matters.
30. **Check the environment variables.** Compare the names and values with `.env.example`.
31. **Check region consistency** for both the web service and the database.
32. **Reproduce locally.** Setting the same environment variables locally usually reproduces the cloud error instantly and turns guesswork into a fast fix.

6.1 Security Practices Applied

6.2 Summary Table

Practice	How It Was Applied	Result
Secrets outside the code	All configuration read with <code>os.getenv()</code>	No password or token exists in the repository
.gitignore protection	Blocks <code>.env</code> , <code>*.pem</code> , <code>*.key</code> , <code>*.db</code> , <code>__pycache__</code> , <code>venv</code>	A secret cannot be committed by accident
Secret masking in logs	<code>mask_database_url()</code> replaces the password with <code>****</code>	Logs can be shared or pasted safely
Least privilege	Database reached only over the private internal network	The database has no public exposure
Strong secret key	64-character value from <code>secrets.token_hex(32)</code>	Session cookies cannot be forged
Input validation	Required fields and length limits checked server-side	Malformed input is rejected before it reaches the database
Output escaping (XSS)	Tags stripped from input; Jinja2 escapes all output	<code><script></code> input is displayed as text, never executed
SQL injection prevention	SQLAlchemy parameterised queries only	No raw SQL is built by string concatenation
TLS in transit	HTTPS provided automatically by the platform	All traffic is encrypted
Error disclosure control	Database errors are logged, not shown to visitors	Internal details are not leaked to the public
MFA on accounts	Multi-factor authentication on the deployment account	A stolen password alone is not enough

6.3 Why a Leaked Secret Cannot Be Un-leaked

Git keeps every version of every file. Deleting a secret in a later commit does not remove it from the history - it remains accessible in the old commit and in any clone. This is why the correct approach is prevention: keep secrets out of the repository in the first place, using environment variables and a working `.gitignore`.

If a secret is ever committed: Revoke and rotate it immediately on the provider, **then** remove it from the code. Rotating first is what actually protects the system; history rewriting afterwards is only tidying.

6.4 Beyond the Project

The following practices were noted during the module but not required by Cloud Notes: enabling multi-factor authentication on every cloud account, using short-lived credentials such as IAM roles instead of stored access keys, encrypting databases at rest, restricting security groups to known source addresses, running dependency scans to catch known vulnerabilities, and turning on audit logging so that any change to configuration is recorded.

7.1 Key Learnings

7.2 About Cloud Services

- 33. **Cloud services are categories, not products.** Compute, storage, databases, networking, identity and monitoring describe *what* a service does; AWS, Azure and Render are simply different providers of the same categories.
- 34. **Managed does not mean free of responsibility.** The provider patches and backs up the infrastructure, but the developer still owns data modelling, access control and cost.
- 35. **Serverless is not always cheaper.** A small Flask application that must answer a request immediately suits a container better than a function, because functions suit short, event-driven work.
- 36. **The network is a design decision.** Placing a database in a private subnet while only the web service is public is the single most effective security control in the whole architecture.

7.3 About Security

- 37. **Environment variables replace configuration files in the cloud.** Twelve-factor applications keep configuration in the environment, which is exactly what a PaaS expects.
- 38. **`.gitignore` is a security control, not a convenience.** It is the mechanism that stops a database password from entering Git history permanently.
- 39. **Least privilege is a habit.** Every policy should be written as explicit allows plus explicit denies, never as a blanket *** on ***.
- 40. **Health checks are a security-adjacent tool.** A monitored `/health` endpoint is how an organisation learns that a service is degraded before a user complains.

7.4 About Deployment

- 41. **The same code runs everywhere.** Only the environment variables changed between the laptop and the cloud, which is the whole point of twelve-factor design.
- 42. **Configuration is the most common cause of deployment failure.** The `postgres://` to `postgresql://` conversion is a small detail that can block an entire deployment.
- 43. **Logs are a platform feature.** Writing to stdout is what makes the Render log dashboard, CloudWatch and Azure Monitor work at all.
- 44. **Free tiers have real limits.** Sleeping instances, expiring databases and metered build minutes must all be planned for.
- 45. **Troubleshooting is a skill.** Build output, then logs, then environment variables is a reliable order that resolves most problems quickly.

7.5 About the Deployment Flow

The required flow - **Application Code** → **Cloud Compute** → **Database / Storage** → **Networking** → **Live Web Application** - proved to be the right mental model. Each stage has a single responsibility, each stage is independently replaceable, and a failure in one stage is diagnosable by looking only at that stage. For example, a `502` error is a compute or start-up problem, while a `503` from `/health` is a database or networking problem. Being able to tell those two apart from the status code alone is the practical value of the model.

8.1 Conclusion

Module 3 achieved both of its objectives. The service categories explored during the module - compute, object storage, managed databases, virtual networking, IAM policies and cloud monitoring - were each studied in terms of purpose, strengths, limitations and cost, and each was related back to the practical project.

The practical outcome is **Cloud Notes**, a Flask application that accepts a name and a note, stores it in a managed PostgreSQL database and lists the latest fifty notes. It reads its entire configuration from environment variables, contains no secret of any kind, writes structured logs to standard output, validates and escapes every piece of user input, and exposes both a `/health` monitoring endpoint and an `/api/notes` JSON endpoint. All 25 automated tests passed, and the application was additionally verified over real HTTP before deployment.

The most valuable outcome is the change in mindset. The same code that ran on a laptop ran unchanged in the cloud, and the only difference was a set of environment variables. This demonstrated that deploying to the cloud is not a special skill requiring infrastructure expertise, but a matter of understanding where configuration, secrets and data belong, and of following a clear, repeatable sequence.

The main practical obstacles encountered were configuration-related rather than code-related: the `postgres://` URL format, matching regions between the web service and the database, and reading the deployment logs to identify a failing start-up. Each was resolved, and the troubleshooting guide in Section 5 records them so the deployment can be repeated without rediscovering them.

For a small team, the next steps would be to move beyond the free tier to avoid instance sleeping, enable automated database backups, add user authentication, and extend the application with object storage for file attachments.

References

- [1] Amazon Web Services. (2024). *Amazon Elastic Compute Cloud (Amazon EC2) Documentation*. <https://docs.aws.amazon.com/ec2/>
- [2] Amazon Web Services. (2024). *Amazon Simple Storage Service (Amazon S3) User Guide*. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/>
- [3] Amazon Web Services. (2024). *Amazon Relational Database Service (Amazon RDS) User Guide*. <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/>
- [4] Amazon Web Services. (2024). *Amazon Virtual Private Cloud (Amazon VPC) Documentation*. <https://docs.aws.amazon.com/vpc/>
- [5] Amazon Web Services. (2024). *Identity and Access Management (IAM) Policies Reference*. https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies.html
- [6] Amazon Web Services. (2024). *Amazon CloudWatch Documentation*. <https://docs.aws.amazon.com/AmazonCloudWatch/latest/>
- [7] Microsoft. (2024). *Azure Virtual Network Documentation*. <https://learn.microsoft.com/azure/virtual-network/>
- [8] Microsoft. (2024). *Azure Database for PostgreSQL Documentation*. <https://learn.microsoft.com/azure/postgresql/>
- [9] Render. (2024). *Render Documentation - Web Services, Databases and Environment Variables*. <https://docs.render.com/>
- [10] Pallets Projects. (2024). *Flask Documentation (2.3.x)*. <https://flask.palletsprojects.com/>
- [11] Pallets Projects. (2024). *Flask-SQLAlchemy Documentation (3.1.x)*. <https://flask-sqlalchemy.palletsprojects.com/>
- [12] Python Software Foundation. (2024). *logging - Logging HOWTO*. <https://docs.python.org/3/howto/logging.html>
- [13] Python Software Foundation. (2024). *venv - Virtual Environments*. <https://docs.python.org/3/library/venv.html>
- [14] Grinberg, A. (2023). *Flask Web Development* (3rd ed.). O'Reilly Media.

- [15] Wigley, A. (2015). *15-Minute Deploy*. Apress.
- [16] W3C. (2024). *HTML Living Standard - Form Submission and Output Escaping*. <https://html.spec.whatwg.org/>
- [17] OWASP. (2024). *Cross-Site Scripting Prevention Cheat Sheet (XSS)*.
https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- [18] UptimeRobot. (2024). *UptimeRobot Documentation*. <https://uptimerobot.com/>

Author Details

Detail	Information
Name	Thajudeen Shuknatha Zainab
GitHub Username	@zainab1315
GitHub Profile	https://github.com/zainab1315
Repository	https://github.com/zainab1315/Module-3-Cloud-Web-App
Live Application	https://cloud-notes-app.onrender.com
Health Endpoint	https://cloud-notes-app.onrender.com/health
JSON API	https://cloud-notes-app.onrender.com/api/notes
Programme	Codomax Digital Solutions - Internship
Module	Module 3 - Cloud Services & Web App Deployment (Day 11-15)
Local Working Copy	D:\Module3_Cloud_Web_App

Declaration

I declare that this report and the accompanying web application are my own work, produced during the Module 3 internship at Codomax Digital Solutions. All configuration values and credentials are supplied through environment variables and are not stored anywhere in the source code or in this document.

Acknowledgement

I would like to thank the trainers and mentors of Codomax Digital Solutions for their guidance throughout Module 3, and in particular for the practical cloud demonstrations that made the deployment flow in this report easy to understand and apply.